Foundations of Autoencoder

[Autoencoder]

1.0 Content Block

Concrete autoencoder (CAE) The concrete autoencoder is designed for discrete feature selection. A concrete autoencoder forces the latent space to consist only of a user-specified number of features. The concrete autoencoder uses a continuous relaxation of the categorical distribution to allow gradients to pass through the feature selector layer, which makes it possible to use standard backpropagation to learn an optimal subset of input features that minimize reconstruction loss.

2.0 Content Block

Definition An autoencoder is defined by the following components: Two sets: the space of encoded messages Z {\displaystyle {\mathcal {Z}}} ; the space of decoded messages X {\displaystyle {\mathcal {X}}} . Typically X {\displaystyle {\mathcal {X}}} and Z {\displaystyle {\mathcal {Z}}} are Euclidean spaces, that is, X = R m , Z = R n {\displaystyle {\mathcal {X}}=\mathbb {R} ^{m},{\mathcal {Z}}=\mathbb {R} ^{n}} with m > n . {\displaystyle m>n.} Two parametrized families of functions: the encoder family E φ : X → Z {\displaystyle E_{\phi }:{\mathcal {X}}\rightarrow {\mathcal {Z}}} , parametrized by φ {\displaystyle \phi } ; the decoder family D θ : Z → X {\displaystyle D_{\theta }:{\mathcal {Z}}\rightarrow {\mathcal {X}}} , parametrized by θ {\displaystyle \theta } .For any x ∈ X {\displaystyle x\in {\mathcal {X}}} , we usually write z = E φ (x) {\displaystyle z=E_{\phi }(x)} , and refer to it as the code, the latent variable, latent representation, latent vector, etc. Conversely, for any z ∈ Z {\displaystyle z\in {\mathcal {Z}}} , we usually write x ′ = D θ (z) {\displaystyle x'=D_{\theta }(z)} , and refer to it as the (decoded) message. Usually, both the encoder and the decoder are defined as multilayer perceptrons (MLPs). For example, a one-layer-MLP encoder E φ {\displaystyle E_{\phi }} is:

3.0 Content Block

There exist representations to the messages that are relatively stable and robust to the type of noise we are likely to encounter; The said representations capture structures in the input distribution that are useful for our purposes. Example noise processes include:

4.0 Content Block

Further reading Bank, Dor; Koenigstein, Noam; Giryes, Raja (2023). "Autoencoders". Machine Learning for Data Science Handbook. Cham: Springer International Publishing. doi:10.1007/978-3-031-24628-9_16. ISBN 978-3-031-24627-2. Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron (2016). "14. Autoencoders". Deep learning. Adaptive computation and machine learning. Cambridge, Mass: The MIT press. ISBN 978-0-262-03561-3.

5.0 Content Block

Image processing The characteristics of autoencoders are useful in image processing. One example can be found in lossy image compression, where autoencoders outperformed other approaches and proved competitive against JPEG 2000. Another useful application of autoencoders in image preprocessing is image denoising. Autoencoders found use in more demanding contexts such as medical imaging where they have been used for image denoising as well as super-resolution. In image-assisted diagnosis, experiments have applied autoencoders for breast cancer detection and for modelling the relation between the cognitive decline of Alzheimer's disease and the latent features of an autoencoder trained with MRI.

6.0 Content Block

Anomaly detection Another application for autoencoders is anomaly detection. By learning to replicate the most salient features in the training data under some of the constraints described previously, the model is encouraged to learn to precisely reproduce the most frequently observed characteristics. When facing anomalies, the model should worsen its reconstruction performance. In most cases, only data with normal instances are used to train the autoencoder; in others, the frequency of anomalies is small compared to the observation set so that its contribution to the learned representation could be ignored. After training, the autoencoder will accurately reconstruct "normal" data, while failing to do so with unfamiliar anomalous data. Reconstruction error (the error between the original data and its low dimensional reconstruction) is used as an anomaly score to detect anomalies. Typically, this means that on a validation set the empirical distribution of reconstruction errors is recorded and then (e.g.) the empirical 95-percentile x p {\displaystyle x_{p}} is taken as threshold t := x p {\displaystyle t:=x_{p}} to flag anomalous data points: loss (x , reconstruction (x)) > t ■ anomaly {\displaystyle {\text{loss}}(x,{\text{reconstruction}}(x))>t\implies {\text{anomaly}}} . Since the threshold is an empirical quantile estimate, there is an inherent difficulty with "correctly" setting this threshold: In many cases the distribution of the empirical quantile is asymptotically a normal distribution empirical p-quantile ~ N (μ = p , σ 2 = p (1 − p) n f (x p) 2) , {\displaystyle {\text{empirical p-quantile}}\sim {\mathcal {N}}\left(\mu =p,\sigma ^{2}={\frac {p(1-p)}{nf(x_{p})^{2}}}\right),} with f (x p) {\displaystyle f(x_{p})} the probability density at the quantile. This means that the variance grows if an extreme quantile is considered (because f (x p) {\displaystyle f(x_{p})} is small there). This means that there is a, potentially, a big uncertainty in what is the right choice for the threshold since it is estimated from a validation set. Recent literature has however shown that certain autoencoding models can, counterintuitively, be very good at reconstructing anomalous examples and consequently not able to reliably perform anomaly detection. Intuitively, this can be understood by considering those one layer auto encoders which are related to PCA - also in this case there can be perfect rein reconstructions for points which are far away from the data region but which lie on a principal component axis. It is best to analyze if the anomalies which are flagged by the auto encoder are true anomalies. In this sense all the metrics in Evaluation of binary classifiers can be considered. The fundamental challenge which comes with the unsupervised (self-supervised) learning setting is, that labels for rare events do not exist (in which case the labels first have to be gathered and the data set will be imbalanced) or anomaly indicating labels are very rare, introducing larger confidence intervals for these performance estimates.

7.0 Content Block

History (Oja, 1982) noted that PCA is equivalent to a neural network with one hidden layer with identity activation function. In the language of autoencoding, the input-to-hidden module is the encoder, and the hidden-to-output module is the decoder. Subsequently, in (Baldi and Hornik, 1989) and (Kramer, 1991) generalized PCA to autoencoders, a technique which they termed "nonlinear PCA". Immediately after the resurgence of neural networks in the 1980s, it was suggested in 1986 that a neural network be put in "auto-association mode". This was then implemented in (Harrison, 1987) and (Elman, Zipser, 1988) for speech and in (Cottrell, Munro, Zipser, 1987) for images. In (Hinton, Salakhutdinov, 2006), deep belief networks were developed. These train a pair restricted Boltzmann machines as encoder-decoder pairs, then train another pair on the latent representation of the first pair, and so on. The first applications of AE date to early 1990s. Their most traditional application was dimensionality reduction or feature learning, but the concept became widely used for learning generative models of data. Some of the most powerful AIs in the 2010s involved autoencoder modules as a component of larger AI systems, such as VAE in Stable Diffusion, discrete VAE in Transformer-based image generators like DALL-E 1, etc. During the early days, when the terminology was uncertain, the autoencoder has also been called identity mapping, auto-associating, self-supervised backpropagation, or Diabolo network.